MCTR911 – Robotics Programming

Milestone 2 Report

Frame Assignment, DH Convention and Kinematics of the UR5e, and Linking ROS2 with MuJoCo

Team 5

Name	ID
Mahmoud Ghobashy	58-2432
Kevin Ezzat	58-0408
Seif Hassan	58-6034
Mohamed Elsayed	58-16392
Fawzy Karim	58-9263

German University in Cairo – Mechatronics Department – Winter 2026

github.com/Mabrouukk/MCTR911-Robotics-Programming

1. Introduction

In Milestone 1 we chose a colour sorting application for a Universal Robots UR5e arm with a Robotiq 2F-85 gripper: parts of different colours arrive on a conveyor, and the robot picks each one up and drops it into the bin of its colour. We also got ROS2 and MuJoCo running. In this milestone we:

- assigned coordinate frames to the arm and built its Denavit-Hartenberg (DH) table,
- wrote the forward and inverse position kinematics in Python and checked them against MuJoCo,
- linked ROS2 to MuJoCo, so that joint commands sent from ROS2 move the simulated robot and the robot's state comes back into ROS2,
- drove the joints with constants, signal builders and sliders, and
- built the first version of the colour sorting cell (a moving conveyor, a colour sensor at the pick station and three colour bins) and ran the whole sorting cycle on it using our inverse kinematics.

All the code is in the *Milestone 02/code* folder of our repository, and this report and its figures are in *Milestone 02/report*. The figures and numbers below come from our own scripts (*fk_analysis.py* and *render_scene.py*), so anyone can re-run them and get the same results.

2. Frame assignment

2.1 The rules we followed

We used the standard DH convention (Denavit and Hartenberg, 1955), the same one used in Craig's textbook and in Universal Robots' own documentation:

- z_{i-1} lies along the rotation axis of joint i.
- x_i lies along the common normal between z_{i-1} and z_i, pointing away from z_{i-1}.
- y_i completes a right-handed frame.
- Frame {0} is fixed at the robot base with z_0 pointing up. Frame {6} is at the tool flange, which is exactly where the gripper is mounted.

With these rules each link is described by four numbers: the joint angle θ_i, the offset d_i along z_{i-1}, the link length a_i along x_i, and the twist α_i about x_i.

2.2 Reading the link lengths from the simulator

Instead of copying numbers from a datasheet, we took the lengths straight from the MuJoCo model we simulate (the UR5e from Google DeepMind's *mujoco_menagerie*). MuJoCo places a frame at every joint (Figure 1). These are not DH frames, since the model's author placed them in whatever way was convenient, but the offsets between them give us the link lengths directly (Table 1).

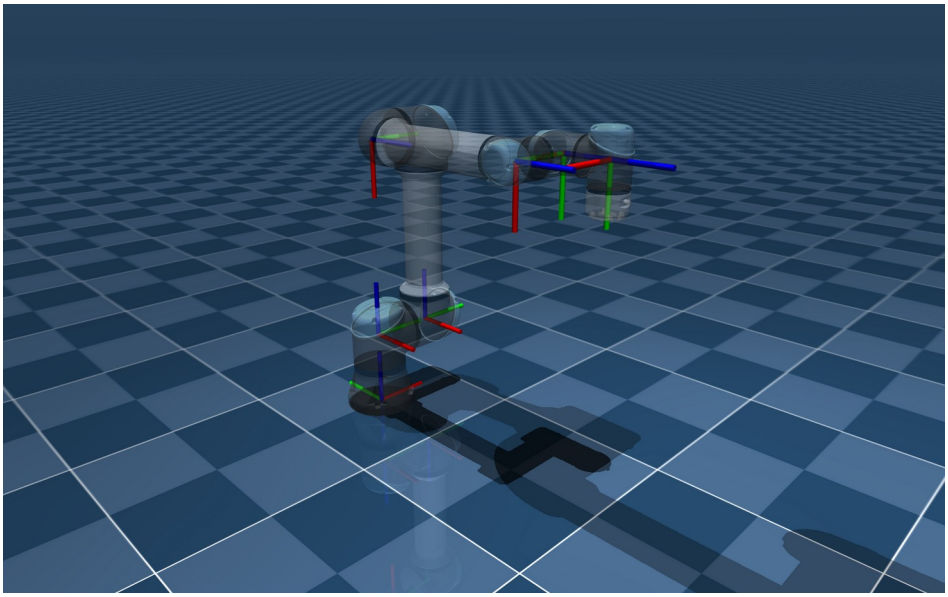


Figure 1: The joint frames that MuJoCo defines for the UR5e (arm drawn see-through). x: red, y: green, z: blue.

DH length	Value	Where it comes from in the model file (ur5e.xml)
d_1	0.163 m	height of the shoulder above the base (shoulder_link pos = 0 0 0.163)
a_2	0.425 m	length of the upper arm (forearm_link pos, z = 0.425)
a_3	0.392 m	length of the forearm (wrist_1_link pos, z = 0.392)
d_4	0.134 m	the three sideways offsets along the parallel joint axes: 0.138 − 0.131 + 0.127
d_5	0.100 m	wrist 2 to wrist 3 (wrist_3_link pos, z = 0.1)
d_6	0.100 m	wrist 3 to the flange where the gripper mounts (attachment_site, 0.1)

Table 1: Link lengths read from the MuJoCo model.

2.3 Our DH frames

Figure 2 shows the DH frames we assigned, drawn by *fk_analysis.py* directly from the DH table, with every joint at zero. In this configuration the UR5e lies stretched out horizontally, which is the usual way to show its frames. The right panel zooms in on the wrist, where frames {3} to {6} are only about 10 cm apart.

A useful thing to notice: z_1, z_2 and z_3 (the axes of joints 2, 3 and 4) are all parallel. So the shoulder, elbow and first wrist joint act like a flat, planar three-link arm, which makes both the forward and inverse kinematics much simpler.

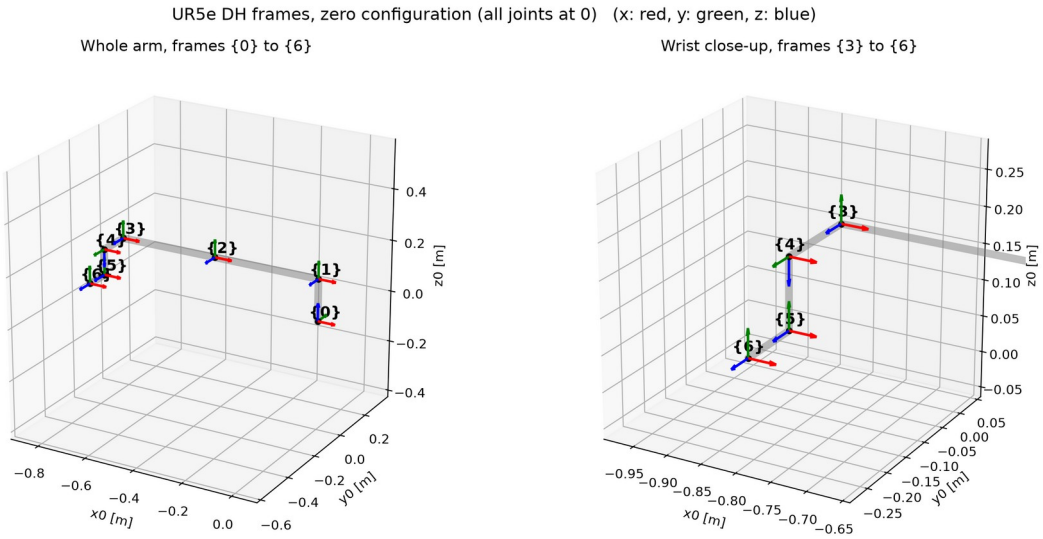


Figure 2: DH frames of the UR5e at the zero configuration (x: red, y: green, z: blue).

3. DH table

Link i	θ_i	d_i (m)	a_i (m)	α_i
1	q_1	0.163	0	+90°
2	q_2	0	−0.425	0°
3	q_3	0	−0.392	0°
4	q_4	0.134	0	+90°
5	q_5	0.100	0	−90°
6	q_6	0.100	0	0°

Table 2: DH parameters of the UR5e (standard convention). All six joints are revolute, so $\theta_i = q_i$ is the variable.

a_2 and a_3 are negative because, with x_i chosen as above, the upper arm and forearm point along −x at the zero pose (you can see this in Figure 2). Universal Robots uses the same signs in their published table.

Table 3 compares our values with the official UR5e datasheet. The MuJoCo model rounds the lengths to the millimetre. We kept the model's values so that our equations describe exactly the robot being simulated; every difference is under 1 mm.

Parameter	Our model	Official UR5e	Difference
d_1	0.163	0.1625	0.5 mm
a_2	−0.425	−0.425	0
a_3	−0.392	−0.3922	0.2 mm
d_4	0.134	0.1333	0.7 mm
d_5	0.100	0.0997	0.3 mm
d_6	0.100	0.0996	0.4 mm

Table 3: Our DH lengths compared with Universal Robots' published values (metres).

4. Forward kinematics

4.1 One link at a time

Each row of the DH table becomes one 4×4 homogeneous transform. It rotates by θ_i about z, slides d_i along z, slides a_i along x, then twists by α_i about x:

$$A_i = \mathrm{Rot}_z(\theta_i) \cdot \mathrm{Trans}_z(d_i) \cdot \mathrm{Trans}_x(a_i) \cdot \mathrm{Rot}_x(\alpha_i)$$

$$\begin{vmatrix} \cos\theta_i & -\sin\theta_i \cos\alpha_i & \sin\theta_i \sin\alpha_i & a_i \cos\theta_i \\ \sin\theta_i & \cos\theta_i \cos\alpha_i & -\cos\theta_i \sin\alpha_i & a_i \sin\theta_i \\ 0 & \sin\alpha_i & \cos\alpha_i & d_i \\ 0 & 0 & 0 & 1 \end{vmatrix}$$

4.2 The whole arm

Multiplying the six link transforms gives the pose of the flange (frame {6}) relative to the base (frame {0}):

$$^0T_6 = A_1\,A_2\,A_3\,A_4\,A_5\,A_6 = \begin{vmatrix} R\ (3\times3) & p\ (3\times1) \\ 0\ \ 0\ \ 0 & 1 \end{vmatrix}$$

Here R is the 3×3 orientation of the gripper and $p = (p_x, p_y, p_z)$ is its position. In our work cell the robot stands on a 0.4 m pedestal, so the position in the world is p + (0, 0, 0.4).

4.3 Closed-form position equations

We multiplied the matrices out symbolically (with SymPy) and simplified the result. Writing $c_i = \cos q_i$, $s_i = \sin q_i$, $c_{23} = \cos(q_2+q_3)$ and $c_{234} = \cos(q_2+q_3+q_4)$ (and the same for sines):

$$p_x = c_1(a_2c_2 + a_3c_{23} + d_5s_{234} - d_6s_5c_{234}) + s_1(d_4 + d_6c_5)$$
$$p_y = s_1(a_2c_2 + a_3c_{23} + d_5s_{234} - d_6s_5c_{234}) - c_1(d_4 + d_6c_5)$$
$$p_z = d_1 + a_2s_2 + a_3s_{23} - d_5c_{234} - d_6s_5s_{234}$$

As expected from the parallel axes in Section 2.3, joints 2, 3 and 4 only show up as sums (q_2+q_3 and $q_2+q_3+q_4$). We checked these three equations against the full matrix product on 200 random poses and they agree to 3×10^{-16} m, which is just rounding.

4.4 Worked example: the home pose

Our home pose is q = (−90°, −90°, 90°, −90°, −90°, 0°). Then $c_1 = 0$, $s_1 = -1$, $s_2 = -1$, $q_2+q_3 = 0$ (so $c_{23} = 1$, $s_{23} = 0$), $q_2+q_3+q_4 = -90°$ (so $c_{234} = 0$, $s_{234} = -1$), $c_5 = 0$ and $s_5 = -1$. Substituting:

$$p_x = 0 + (-1)(0.134 + 0) = -0.134 \text{ m}$$
$$p_y = (-1)(0 - 0.392 - 0.100 - 0) - 0 = 0.492 \text{ m}$$
$$p_z = 0.163 + 0.425 + 0 - 0 - 0.100 = 0.488 \text{ m}$$

The full transform from our Python code agrees:

$$\begin{vmatrix} 1 & 0 & 0 & -0.134 \\ 0 & -1 & 0 & 0.492 \\ 0 & 0 & -1 & 0.488 \\ 0 & 0 & 0 & 1 \end{vmatrix}$$

The rotation part is diag(1, −1, −1), meaning the gripper points straight down with its x-axis along the world x-axis. That's the natural orientation for picking parts off a conveyor, which is why we use this as the robot's home pose. For sorting we turn the gripper a further 90° about the vertical, so that its fingers close across the conveyor belt instead of along it (Section 8).

4.5 Python implementation

The forward kinematics lives in *kinematics.py*. It is a direct translation of Sections 4.1 and 4.2:

```
D     = [0.163, 0, 0, 0.134, 0.1, 0.1]
A     = [0, -0.425, -0.392, 0, 0, 0]
ALPHA = [pi/2, 0, 0, pi/2, -pi/2, 0]

def dh_transform(theta, d, a, alpha):
    ct, st, ca, sa = cos(theta), sin(theta), cos(alpha), sin(alpha)
    return [[ct, -st*ca,  st*sa, a*ct],
            [st,  ct*ca, -ct*sa, a*st],
            [0,   sa,     ca,    d   ],
            [0,   0,      0,     1   ]]

def forward_kinematics(q):
    T = identity(4)
    for i in range(6):
        T = T @ dh_transform(q[i], D[i], A[i], ALPHA[i])
    return T
```

5. Checking the forward kinematics

5.1 Against MuJoCo's own kinematics

MuJoCo computes the position and orientation of every part of the robot internally. So for any set of joint angles, we can compare our DH result with MuJoCo's answer for the flange (the *attachment_site* where the gripper is mounted). *fk_analysis.py* does this for the zero pose, the home pose and three random poses (Table 4).

Pose	Joint angles (°)	Our FK position (m)	MuJoCo position (m)	Position error	Orientation error
zero	0, 0, 0, 0, 0, 0	(−0.817, −0.234, 0.063)	(−0.817, −0.234, 0.063)	2.4×10^{-13} mm	4.4×10^{-16}
home	−90, −90, 90, −90, −90, 0	(−0.134, 0.492, 0.488)	(−0.134, 0.492, 0.488)	1.8×10^{-13} mm	8.9×10^{-16}
random 1	45, 143, 99, −99, −72, 134	(0.4739, 0.2410, 0.3911)	(0.4739, 0.2410, 0.3911)	1.9×10^{-13} mm	4.4×10^{-16}
random 2	−178, 116, 107, −12, −71, −80	(−0.3453, 0.1554, 0.0821)	(−0.3453, 0.1554, 0.0821)	1.1×10^{-13} mm	5.6×10^{-16}
random 3	−88, −20, 2, 19, 178, 105	(−0.0577, 0.7719, 0.3287)	(−0.0577, 0.7719, 0.3287)	4.5×10^{-13} mm	7.8×10^{-16}

Table 4: Our forward kinematics vs. MuJoCo, in the robot base frame. Orientation error is the largest difference between any two rotation-matrix entries.

The errors are around 10^{-13} mm, which is floating-point rounding. So our DH model and MuJoCo agree exactly on where the gripper flange is and how it is oriented.

5.2 Live, through ROS2

We also did the check the way it will be done in Milestone 3: sensing the robot through ROS2 while it runs. We sent the arm to (−0.2, 0.6, 0.55) m in the world, then read the joint angles from */ur5e/joint_states* and the flange position from */ur5e/ee_position*, both coming back from the running simulator:

- sensed joint angles (rad): −1.4625, −0.9719, 1.7544, −2.3212, −1.5708, −1.4625
- sensed flange position: (−0.19959, 0.59624, 0.53447) m
- our FK applied to the sensed angles, plus the pedestal: (−0.19959, 0.59624, 0.53447) m

The two positions agree to 0.000 mm.

5.3 Why the arm stops 1.5 cm low

In that same test the flange reached z = 0.534 m instead of the commanded 0.550 m. This is not a kinematics error, since Section 5.2 shows our FK matches the sensed state exactly. The joint angles themselves are slightly off: the shoulder ended about 1.1° from its command. MuJoCo drives each joint with a simple PD position servo, torque = $K_p(q_{target} - q) - K_d\dot{q}$, with K_p = 2000 and K_d = 400 for the three big joints (500 and 100 for the wrist joints) and no gravity compensation. So the arm settles where the servo torque balances its weight plus the gripper's. Removing this steady-state error is a controller problem, which is what Milestone 4 is about.

6. Inverse kinematics

Inverse kinematics goes the other way: given a desired flange pose ${}^0T_6$, find the joint angles. Because the UR5e has three parallel axes, the problem has a closed-form solution (the approach described by Hawkins, 2013). *inverse_kinematics()* in *kinematics.py* does these steps:

1. **Wrist centre.** Step back d_6 from the flange along the tool z-axis to get the origin of frame {5}: $p_{05} = p - d_6 \cdot z_6$.
2. **q_1.** Frame {5} must sit at a sideways distance d_4 from the plane of joint 1. This gives $q_1 = \text{atan2}(p_{05,y}, p_{05,x}) \pm \text{acos}(d_4 / r) + 90°$, with r the horizontal distance to p_{05}. Two solutions: shoulder left or right.
3. **q_5.** From the position equations, $\cos q_5 = (p_x s_1 - p_y c_1 - d_4) / d_6$. Two solutions: wrist up or down.
4. **q_6.** From the tool orientation: $q_6 = \text{atan2}((-r_{12}s_1 + r_{22}c_1) / s_5, (r_{11}s_1 - r_{21}c_1) / s_5)$. When $s_5 = 0$ the wrist is singular (joints 4 and 6 line up), so we set $q_6 = 0$.
5. **q_2, q_3, q_4.** Remove the known joints 1, 5 and 6 from the target. What's left is the planar three-link arm: q_3 from the law of cosines (elbow up or down), then q_2, and finally $q_4 = (q_2+q_3+q_4) - q_2 - q_3$.

That gives up to 2 × 2 × 2 = 8 solutions. When we use IK to move the robot, we pick the solution closest to the current joint angles and unwrap it, so the arm always turns the short way. For example, when carrying a part from the pick station to the blue bin, joint 1 turns from −113° to

−224° (111°) instead of jumping to the wrapped value +136°, which would be a 249° turn the other way.

Test. We generated 2000 random joint configurations, computed their flange poses with FK, solved IK and checked every solution:

- between 2 and 8 solutions per pose (7 on average),
- the worst $|FK(IK(T)) - T|$ over every solution was 4.4×10^{-13},
- the original joint angles were among the solutions in 2000 out of 2000 cases.

7. Linking ROS2 and MuJoCo

In Milestone 1 we ended up with ROS2 running inside Docker (ROS2 only has proper support on Ubuntu) and MuJoCo running directly on macOS (its 3D viewer could not be shown reliably from inside the container). For this milestone the two had to work as one system, so we connected them with a small network link over Docker's standard *host.docker.internal* address (Figure 3).

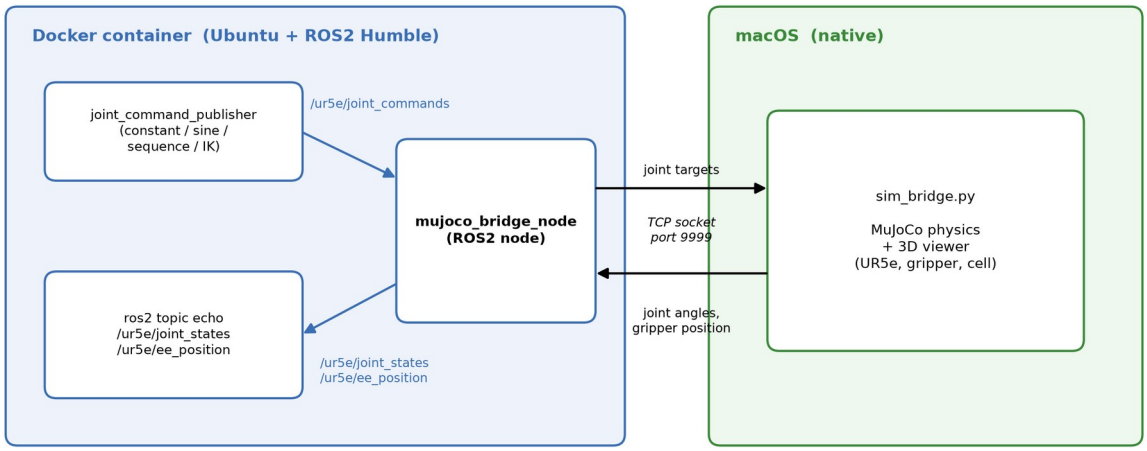


Figure 3: How ROS2 (left, in Docker) and MuJoCo (right, on macOS) are connected.

- **sim_bridge.py** (macOS) builds the scene (arm + gripper + sorting cell), opens the MuJoCo viewer, and listens on TCP port 9999. Each incoming command sets the joint motor targets, clipped to the joint limits. It also runs the conveyor and the colour sensor (Section 9). 30 times a second it sends back the joint angles, the gripper command, the flange position and the colour of the part waiting at the pick station.
- **mujoco_bridge_node.py** (ROS2 node in Docker) connects to that port and turns messages into ROS2 topics in both directions. If the simulator restarts, it reconnects on its own.
- **joint_command_publisher.py** (ROS2 node in Docker) is where the inputs come from (Section 8).

Messages on the socket are one small JSON object per line, for example {"q": [6 joint angles], "gripper": 255} towards the simulator and {"q": [...], "gripper": 0, "ee": [x, y, z], "station": "red"} back from it. We tested the link on its own first (three test messages from the container reached the Mac without problems) before building on it.

Topic	Message type	Direction	Content
/ur5e/joint_commands	std_msgs/	ROS2 → MuJoCo	6 joint angles (rad), optional 7th

Topic	Message type	Direction	Content
	Float64MultiArray		value: gripper (0 = open, 255 = closed)
/ur5e/joint_states	sensor_msgs/JointState	MuJoCo → ROS2	the 6 joint angles measured in the simulation
/ur5e/ee_position	geometry_msgs/ PointStamped	MuJoCo → ROS2	flange position in the world frame (m)
/cell/station_color	std_msgs/String	MuJoCo → ROS2	colour of the part waiting at the pick station: "red", "green", "blue", or "" when empty

Table 5: ROS2 topics used in this milestone.

8. Moving the joints: constants, signal builders and sliders

To analyse and visualise the arm's motion, we can drive the joints in several ways:

- **Constant:** hold one fixed pose, e.g. the home pose or any six angles given in degrees.
- **Signal builder 1, sine wave:** every joint oscillates around home, $q_i(t) = q_{home,i} + 20° \cdot \sin(2\pi \cdot 0.2\ t + 0.5\ i)$. The phase shift makes the motion ripple down the arm, and the gripper opens and closes at the same rate.
- **Signal builder 2, colour sorting sequence:** a waypoint signal that runs our application (Table 6). It waits until the colour sensor reports a part at the pick station, then walks through eight waypoints that carry that part to the bin of its colour. Each waypoint is defined in world coordinates and turned into joint angles with our IK, and the command blends smoothly to it over 0.8 to 2.5 s (a cosine blend, so the arm starts and stops gently), which makes one cycle take 12.8 s. When the cycle ends the robot waits for the next part.
- **Sliders:** the MuJoCo viewer's Control panel has one slider per motor. Whenever no ROS2 input is running, you can drag them to move each joint and the gripper by hand.
- **IK target:** give an (x, y, z) point in the world and the arm moves its flange there with the gripper pointing down.

Step	Waypoint	Flange position in the world (m)	Gripper
1	above station	(0.105, 0.60, 0.70)	open
2	down to part	(0.105, 0.60, 0.486)	open
3	close gripper	(0.105, 0.60, 0.486)	closed
4	lift	(0.105, 0.60, 0.70)	closed
5	above the bin of the part's colour	red (0.55, 0.25, 0.70), green (0.55, −0.05, 0.70), blue (0.55, −0.35, 0.70)	closed
6	down into the bin	same x, y as step 5, z = 0.52	closed
7	open gripper (part drops in)	same as step 6	open
8	back above station	(0.105, 0.60, 0.70)	open

Table 6: One sorting cycle. The grasp height puts the gripper pads just above the centre of the 5 cm part, and the drop height releases the part a few centimetres above the bin's rim.

While any of these runs, */ur5e/joint_states*, */ur5e/ee_position* and /cell/station_color can be watched live with *ros2 topic echo*, which is how we recorded the numbers in Section 5.2.

9. The colour sorting cell (first version of the GUI)

We built the environment around the robot in its own file, *environment.xml*, which *sim_bridge.py* attaches to the robot scene when it starts (Figures 4 and 5):

- a 0.4 m steel pedestal, with the UR5e mounted on top,
- a 1.2 m accumulating conveyor in front of the robot (belt surface at 0.30 m, with rollers, side rails and legs). The belt carries the parts at 8 cm/s towards a yellow end stop. Parts that reach the end stop, or the part in front of them, stop and queue up, like on a real accumulating conveyor. The belt geom itself does not move; every simulation step, sim_bridge.py gives the belt's speed to each part resting on it,
- a pick station at the end stop, where the front part waits for the robot,
- a colour sensor on a post beside the pick station. It reports the colour of the part waiting there, which reaches ROS2 as /cell/station_color,
- three open bins beside the robot, coloured red, green and blue,
- six 5 cm parts (two of each colour, 0.1 kg each) on the belt in mixed order. They are free bodies, so they collide, are grasped by the gripper's fingers and fall into the bins under real physics,
- yellow safety markings on the floor around the cell.

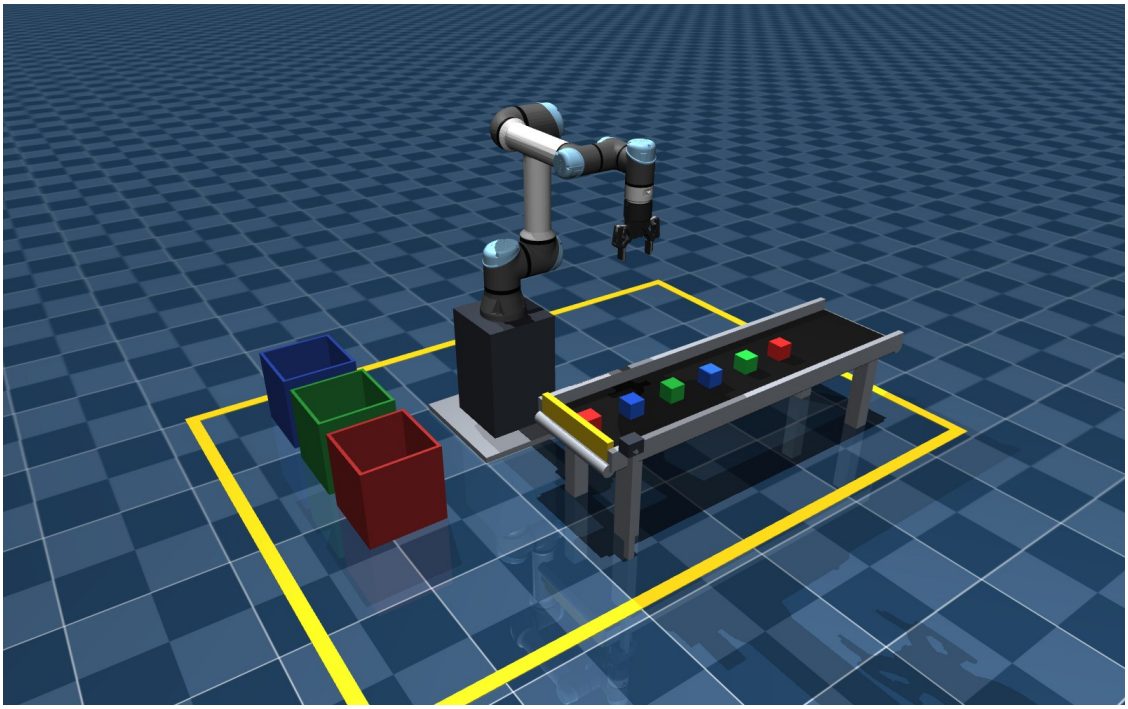


Figure 4: The colour sorting cell in MuJoCo. The first part has reached the pick station and the others are on their way.

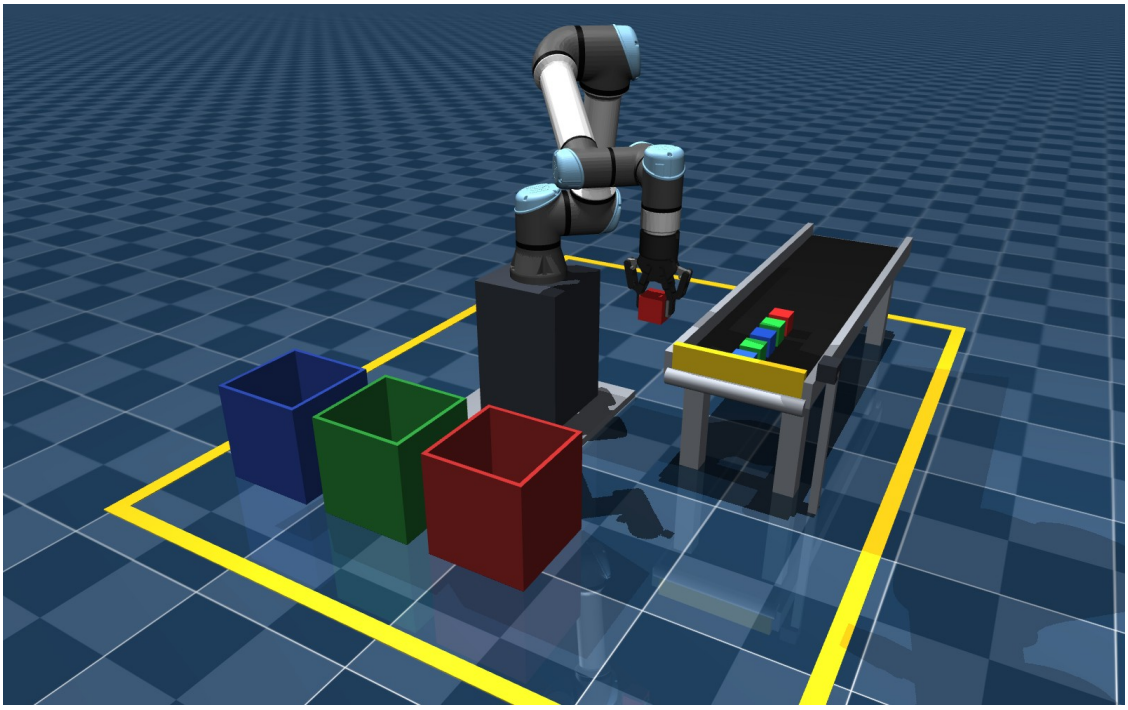


Figure 5: Halfway through the first cycle: the robot carries the red part to the red bin, while the next parts queue at the end stop.

We tested the whole cell end to end, with sim_bridge.py, the ROS2 bridge node and the sorting sequence running together: all six parts were picked from the station and dropped into the bin of their own colour, in about 83 s. For Milestone 3 we plan to add more parts, a reject bin for defective parts and more details of the cell (a worker, a control cabinet), and grow this into the full environment.

10. How to run it

The whole system starts with one command, *scripts/run_ms2.sh*. It opens the simulator, starts the ROS2 bridge in Docker, waits until the two are connected, and then runs the chosen input. Ctrl+C stops everything. The same runs are available in VS Code under ⌘⇧P → "Tasks: Run Task" → "Run Milestone 2".

Command	Input driving the joints
scripts/run_ms2.sh	colour sorting (default)
scripts/run_ms2.sh sine	sine wave on every joint
scripts/run_ms2.sh constant	hold the home pose (or constant --deg q1 … q6)
scripts/run_ms2.sh ik --xyz X Y Z	move the flange to a world point
scripts/run_ms2.sh sliders	no ROS2 input; drag the viewer's Control-panel sliders

Table 7: Running the Milestone 2 system.

python3 fk_analysis.py (in *Milestone 02/code*) re-runs every check in Sections 5 and 6 and redraws Figure 2, and python3 render_scene.py redraws Figures 1, 4 and 5.

11. Summary

We assigned DH frames to the UR5e, built its DH table from the simulated model, and derived its forward kinematics, both as a matrix product and as closed-form position equations. The forward kinematics matches MuJoCo exactly, both offline and live through ROS2. We also have a closed-form inverse kinematics that recovered the original angles in all 2000 random tests. ROS2 and MuJoCo now work as one system: ROS2 commands move the simulated robot and its state flows back into ROS2. The joints can be driven with constants, two kinds of signal builders, sliders and IK targets, and the first version of our colour sorting cell works: a moving conveyor feeds parts to a pick station, a colour sensor reports each part's colour through ROS2, and the robot sorts every part into the bin of its colour.

Next, in Milestone 3, we will add velocity and acceleration kinematics, validate them against the simulator, and complete the environment.

References

1. J. Denavit and R. S. Hartenberg, "A kinematic notation for lower-pair mechanisms based on matrices," *Journal of Applied Mechanics*, 1955.
2. J. J. Craig, *Introduction to Robotics: Mechanics and Control*, Pearson.
3. M. W. Spong, S. Hutchinson and M. Vidyasagar, *Robot Modeling and Control*, Wiley.
4. K. P. Hawkins, "Analytic Inverse Kinematics for the Universal Robots UR-5/UR-10 Arms," Georgia Institute of Technology, 2013.
5. Universal Robots, "DH parameters for calculations of kinematics and dynamics," universal-robots.com.
6. E. Todorov, T. Erez and Y. Tassa, "MuJoCo: A physics engine for model-based control," IROS 2012.
7. Google DeepMind, *MuJoCo Menagerie*, github.com/google-deepmind/mujoco_menagerie.
8. S. Macenski et al., "Robot Operating System 2: Design, architecture, and uses in the wild," *Science Robotics*, 2022.